

Module 1: Angular Environment Setup and CLI

1.2 Installation and Setup

Prerequisites for Angular Development

Before installing Angular, you need to set up the foundational tools that Angular depends on. These prerequisites are essential for Angular to function properly.

Node.js and npm

Node.js is a JavaScript runtime environment that allows JavaScript to run outside the browser. Angular CLI and build tools require Node.js to function.

npm (Node Package Manager) comes bundled with Node.js and is used to install Angular and other dependencies.

Checking if Node.js is Already Installed

Open your terminal or command prompt and run:

```
node --version  
npm --version
```

Output:

```
v18.17.0  
9.8.1
```

Explanation: If you see version numbers, Node.js and npm are already installed. Angular requires Node.js version 14.x or higher.

Installing Node.js

If Node.js is not installed or you have an older version:

1. Visit the official Node.js website: <https://nodejs.org/>
2. Download the **LTS (Long Term Support)** version for your operating system
3. Run the installer and follow the installation wizard
4. Verify installation by running the version commands above

Recommendation: Always use the LTS version for stability and long-term support. Avoid the "Current" version for production projects.

Installing Angular CLI

The Angular CLI (Command Line Interface) is the official tool for creating and managing Angular applications. It must be installed globally on your system to be accessible from any directory.

Global Installation Command

```
npm install -g @angular/cli
```

Output:

```
npm WARN deprecated [packages...]  
added 250 packages in 45s
```

```
25 packages are looking for funding  
run `npm fund` for details
```

Explanation: The `-g` flag installs Angular CLI globally, making the `ng` command available system-wide. Installation may take a few minutes depending on your internet speed.

Understanding the Command

- `npm install` - npm command to install packages
- `-g` - Global flag (installs for all projects, not just current directory)
- `@angular/cli` - The official Angular CLI package name

Verifying Angular CLI Installation

```
ng version
```

Output:



```
Angular CLI: 16.2.0
Node: 18.17.0
Package Manager: npm 9.8.1
OS: win32 x64
```

Explanation: This displays the Angular CLI version along with Node.js and system information, confirming successful installation.

⚠ Important:

If you receive an error like "ng is not recognized as an internal or external command", you may need to:

- Close and reopen your terminal/command prompt

- Check that npm's global bin directory is in your system PATH
- On Windows, restart your computer if the issue persists

Creating Your First Angular Application

With Angular CLI installed, creating a new Angular project is straightforward. The CLI handles all configuration and setup automatically.

Basic Project Creation

```
ng new my-first-app
```

Output:

```
? Would you like to add Angular routing? (y/N) y
? Which stylesheet format would you like to use?
  CSS
  > SCSS   [ https://sass-lang.com/documentation/syntax#scss ]
    Sass   [ https://sass-lang.com/documentation/syntax#the-indented-syntax ]
    Less   [ http://lesscss.org ]

CREATE my-first-app/README.md (1070 bytes)
CREATE my-first-app/.editorconfig (274 bytes)
CREATE my-first-app/.gitignore (548 bytes)
...
✓ Packages installed successfully.
Successfully initialized git.
```

Explanation: The CLI prompts you with configuration questions and then generates all necessary files and folders for your Angular project.

Understanding CLI Prompts

During project creation, Angular CLI asks several configuration questions:

Prompt	Purpose	Recommendation
Add Angular routing?	Sets up routing module for navigation between pages	Select Yes for multi-page applications
Stylesheet format?	Choose CSS preprocessor for styling	CSS for simplicity, SCSS for advanced projects
Server-Side Rendering (SSR)?	Enable server-side pre-rendering of pages	Select No for learning, Yes for SEO-critical apps

Understanding Stylesheet Format Options

Angular CLI offers multiple stylesheet options during project creation. Each has different capabilities and use cases:

Format	File Extension	Description	Best For
CSS	<code>.css</code>	Standard Cascading Style Sheets - plain CSS without preprocessing	Beginners, simple projects, standard web styling
SCSS	<code>.scss</code>	Sassy CSS - Sass with CSS-like syntax, supports variables, nesting, mixins	Most popular choice, large projects, team environments
Sass	<code>.sass</code>	Syntactically Awesome Style Sheets - indentation-based syntax (no braces)	Developers who prefer cleaner, minimal syntax
Less	<code>.less</code>		

Format	File Extension	Description	Best For
		Leaner Style Sheets - CSS preprocessor with variables and mixins	Projects migrating from Bootstrap 3 or Less-based systems

CSS Preprocessor Features Comparison

Plain CSS:

```

/* Standard CSS - No preprocessing */
.button {
  background-color: #3498db;
  color: white;
  padding: 10px 20px;
}

.button:hover {
  background-color: #2980b9;
}

```

SCSS (Recommended):

```

/* SCSS - Variables, nesting, and mixins */
$primary-color: #3498db;
$hover-color: #2980b9;

.button {
  background-color: $primary-color;
  color: white;
  padding: 10px 20px;

  &:hover {
    background-color: $hover-color;
  }
}

```

Output:

Compiled Output: Both examples produce identical CSS. SCSS offers better organization through variables and nesting, making styles easier to maintain.

Sass (Indented Syntax):

```
/* Sass - No braces or semicolons */
$primary-color: #3498db
$hover-color: #2980b9

.button
  background-color: $primary-color
  color: white
  padding: 10px 20px

  &:hover
    background-color: $hover-color
```

Less:

```
/* Less - Similar to SCSS */
@primary-color: #3498db;
@hover-color: #2980b9;

.button {
  background-color: @primary-color;
  color: white;
  padding: 10px 20px;

  &:hover {
    background-color: @hover-color;
  }
}
```

Key Benefits of CSS Preprocessors

- **Variables** - Store colors, fonts, and sizes for reuse across stylesheets
- **Nesting** - Write hierarchical CSS that mirrors HTML structure
- **Mixins** - Create reusable groups of CSS declarations

- **Functions** - Perform calculations and manipulate values
- **Imports** - Split styles into multiple files and combine them
- **Inheritance** - Share properties between selectors

Recommendation for Beginners: Start with **CSS** if you're new to web development. Once comfortable, switch to **SCSS** as it's the most widely used in Angular projects and maintains CSS-compatible syntax.

Understanding Server-Side Rendering (SSR)

Starting with Angular 17+, the CLI asks whether you want to enable Server-Side Rendering (SSR) during project creation. Understanding SSR helps you make an informed decision.

What is Server-Side Rendering?

Server-Side Rendering means that Angular renders your application on the server before sending HTML to the browser, rather than sending JavaScript that renders in the browser.

Traditional SPA Flow (Client-Side Rendering)

1. Browser requests page
↓
2. Server sends minimal HTML + JavaScript bundles
↓
3. Browser downloads JavaScript
↓
4. Angular bootstraps and renders in browser
↓
5. User sees content (may take 1-3 seconds)

Output:

Browser receives:

```
<app-root></app-root>  
<script src="main.js"></script>
```


Issue: Search engines and social media crawlers may not wait for JavaScript to execute, resulting in poor SEO.

SSR Flow (Server-Side Rendering)

1. Browser requests page
↓
2. Server runs Angular and generates complete HTML
↓
3. Server sends fully-rendered HTML to browser
↓
4. User sees content immediately (faster)
↓
5. JavaScript downloads and "hydrates" the page
↓
6. Page becomes interactive

Output:

Browser receives:

```
<app-root>
  <h1>Welcome to My App</h1>
  <p>Fully rendered content here...</p>
</app-root>
<script src="main.js"></script>
```

Benefit: Content is visible immediately. Search engines can index the complete HTML.

SSR vs Traditional Multi-Page Apps vs Client-Side SPAs

Aspect	Traditional Multi-Page	Client-Side SPA	SSR SPA
Initial Load			

Aspect	Traditional Multi-Page	Client-Side SPA	SSR SPA
	Fast (server-rendered HTML)	Slow (wait for JS to render)	Fast (server-rendered HTML)
Navigation	Slow (full page reload)	Fast (client-side routing)	Fast (client-side routing)
SEO	Excellent	Poor (requires workarounds)	Excellent
Server Load	High (every request)	Low (static files only)	Medium (initial render only)
User Experience	Page flickers	Smooth, app-like	Smooth, app-like
Complexity	Simple	Moderate	High (requires Node.js server)

Why Use SSR?

Benefits of Server-Side Rendering:

1. Improved SEO (Search Engine Optimization)

- Search engines receive fully-rendered HTML
- Better indexing for Google, Bing, etc.
- Essential for content-heavy websites

2. Better Social Media Sharing

- Facebook, Twitter, LinkedIn can extract page metadata
- Proper preview cards with images and descriptions

3. Faster Initial Page Load

- Users see content before JavaScript downloads
- Improved perceived performance
- Better experience on slow networks

4. Accessibility

- Content available even if JavaScript fails
- Better for users with JavaScript disabled

Drawbacks of SSR:

- **Increased Complexity** - Requires Node.js server to run Angular on the backend
- **Server Resources** - More CPU usage on server for rendering
- **Development Complexity** - Some browser APIs unavailable on server (window, document)
- **Hosting Costs** - Need Node.js hosting instead of static file hosting

When to Use SSR?

Use SSR	Skip SSR
Public-facing websites (blogs, news, e-commerce)	Internal admin dashboards
Content-heavy applications needing SEO	Applications behind login/authentication
Landing pages for marketing	Tools and utilities (calculators, editors)
Portfolio and business websites	Learning projects and prototypes
Applications requiring social media previews	Real-time collaborative applications

For Learning Angular: Select **No** for SSR. Focus on mastering Angular basics first. SSR adds complexity that can be overwhelming for beginners. You can always add SSR later using `ng add @angular/ssr`.

SSR Project Creation Example

```
ng new my-ssr-app
```

Output:

```
? Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? (y/N) n
```

```
Skipping SSR setup. You can add it later with: ng add @angular/ssr
```

Result: Project created without SSR. You can always enable it later when needed.

Creating a Project with Flags (Skip Prompts)

You can skip interactive prompts by providing flags directly:

```
ng new my-app --routing --style=css --ssr=false --skip-git
```

Output:

```
CREATE my-app/angular.json (2876 bytes)
CREATE my-app/package.json (1042 bytes)
CREATE my-app/tsconfig.json (901 bytes)
...
✓ Packages installed successfully.
```

Explanation: This creates a project with routing enabled, CSS for styles, SSR disabled, and skips Git initialization—all without user prompts.

Common ng new Flags

Flag	Description	Example
<code>--routing</code>	Add routing module	<code>ng new app --routing</code>
<code>--style</code>	Stylesheet format (css, scss, sass, less)	<code>ng new app --style=scss</code>

Flag	Description	Example
<code>--skip-git</code>	Skip Git repository initialization	<code>ng new app --skip-git</code>
<code>--skip-install</code>	Skip npm package installation	<code>ng new app --skip-install</code>
<code>--prefix</code>	Custom component selector prefix	<code>ng new app --prefix=my</code>
<code>--ssr</code>	Enable/disable Server-Side Rendering	<code>ng new app --ssr=false</code>

Compiling and Serving the Application

After creating your project, the next step is to run it. Angular CLI provides commands to serve the application locally and build it for production.

Starting the Development Server

Navigate into your project directory and start the server:

```
cd my-first-app
ng serve
```

Output:

✓ Browser application bundle generation complete.

Initial Chunk Files	Names	Raw Size
vendor.js	vendor	2.35 MB
polyfills.js	polyfills	314.27 kB
styles.css, styles.js	styles	210.09 kB
main.js	main	50.83 kB
runtime.js	runtime	6.51 kB
	Initial Total	2.92 MB

```
Build at: 2026-01-23T10:30:45.678Z - Hash: a1b2c3d4e5f6
** Angular Live Development Server is listening on localhost:4200 **
✓ Compiled successfully.
```

Explanation: The development server compiles your application and makes it available at `http://localhost:4200`. The compilation includes all necessary bundles.

Opening in Browser

You can manually open your browser and navigate to:

```
http://localhost:4200
```

Or use the `--open` flag to automatically open the browser:

```
ng serve --open
```

Output:

Result: Browser automatically opens and displays the default Angular welcome page with the Angular logo and helpful links.

Additional ng serve Options

Flag	Description	Example
<code>--open</code> or <code>-o</code>	Automatically open browser	<code>ng serve --open</code>
<code>--port</code>	Change port number	<code>ng serve --port 4300</code>
<code>--live-reload</code>	Enable/disable live reload (default: true)	<code>ng serve --live-reload=false</code>
	Serve using HTTPS	

Flag	Description	Example
<code>--ssl</code>		<code>ng serve --ssl</code>

Building for Production

When you're ready to deploy your application, use the build command:

```
ng build
```

Output:

- ✓ Browser application bundle generation complete.
- ✓ Copying assets complete.
- ✓ Index html generation complete.

Initial Chunk Files	Names	Raw Size	Estimated Transfer Size
main.a1b2c3d4e5f6.js	main	245.67 kB	67.89 kB
polyfills.f6e5d4c3b2a1.js	polyfills	35.12 kB	11.23 kB
runtime.1234567890ab.js	runtime	1.05 kB	600 bytes
styles.abcdef123456.css	styles	0 bytes	0 bytes
Initial Total		281.84 kB	79.72 kB

Build at: 2026-01-23T11:00:15.432Z - Hash: xyz789abc123

Output location: dist/my-first-app/

- ✓ Build complete

Explanation: The build process creates optimized, production-ready files in the `dist/` folder. Files are minified, bundled, and hash-named for cache busting.

Production Build

For production deployments with maximum optimization:

```
ng build --configuration production
```

Production builds include:

- **Ahead-of-Time (AOT) Compilation** - Templates compiled during build
- **Minification** - Code size reduced
- **Tree Shaking** - Unused code removed
- **Bundling** - Files combined for fewer HTTP requests

Angular Application Bootstrapping Process

Understanding how Angular starts up is crucial for troubleshooting and customizing your application. The bootstrapping process follows a specific sequence.

The Bootstrapping Flow

1. index.html (Entry Point)
↓
2. main.ts (Bootstrap Configuration)
↓
3. AppModule (Root Module)
↓
4. AppComponent (Root Component)
↓
5. Application Rendered

1. index.html - The Entry Point

Located at `src/index.html`, this is the single HTML page that hosts your Angular application:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>MyFirstApp</title>
  <base href="/">
```



```
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

Output:

Key Elements:

- `<base href="/">` - Sets the base URL for relative paths and routing
- `<app-root></app-root>` - Custom HTML tag where Angular application will be loaded
- No script tags visible - Angular CLI automatically injects required JavaScript bundles during build

2. main.ts - Bootstrap Configuration

Located at `src/main.ts`, this TypeScript file bootstraps the Angular application:

```
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

Output:

Explanation:

- `platformBrowserDynamic()` - Creates a platform for running Angular in the browser with JIT (Just-in-Time) compilation
- `bootstrapModule(AppModule)` - Initializes the root module of your application

- `.catch()` - Handles any bootstrap errors

3. Polyfills (Browser Compatibility)

The `polyfills.ts` file (if present) loads compatibility features for older browsers:

```
// Polyfills for older browsers
import 'zone.js'; // Required by Angular
```

Output:

Purpose: `zone.js` is essential for Angular's change detection mechanism. It patches asynchronous operations to track when to update the view.

Understanding Angular Application Structure

An Angular project created with CLI has a well-organized structure. Understanding this structure helps you navigate and maintain your application effectively.

Project Root Structure

```
my-first-app/
|
├─ node_modules/      # npm packages (dependencies)
├─ src/               # Source code
├─ .editorconfig      # Editor configuration
├─ .gitignore         # Git ignore rules
├─ angular.json       # Angular CLI configuration
├─ package.json       # npm dependencies and scripts
├─ tsconfig.json      # TypeScript configuration
├─ tsconfig.app.json  # TypeScript config for application
└─ tsconfig.spec.json # TypeScript config for tests
```

Key Configuration Files

1. package.json

Manages project dependencies and npm scripts:

```
{
  "name": "my-first-app",
  "version": "0.0.0",
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    "build": "ng build",
    "test": "ng test"
  },
  "dependencies": {
    "@angular/core": "^16.2.0",
    "@angular/common": "^16.2.0",
    "@angular/platform-browser": "^16.2.0",
    "rxjs": "~7.8.0",
    "zone.js": "~0.13.0"
  },
  "devDependencies": {
    "@angular/cli": "~16.2.0",
    "typescript": "~5.1.3"
  }
}
```

Output:

Key Sections:

- `scripts` - npm commands you can run (e.g., `npm start` runs `ng serve`)
- `dependencies` - Packages required for the application to run
- `devDependencies` - Packages only needed during development

2. angular.json

Central configuration file for Angular CLI:

```
{
  "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
  "version": 1,
  "newProjectRoot": "projects",
  "projects": {
    "my-first-app": {
      "projectType": "application",
      "root": "",
      "sourceRoot": "src",
      "architect": {
        "build": {
          "builder": "@angular-devkit/build-angular:browser",
          "options": {
            "outputPath": "dist/my-first-app",
            "index": "src/index.html",
            "main": "src/main.ts",
            "polyfills": ["zone.js"],
            "assets": ["src/favicon.ico", "src/assets"],
            "styles": ["src/styles.css"],
            "scripts": []
          }
        }
      }
    }
  }
}
```

Output:

Purpose: Configures build settings, file locations, styles, scripts, and assets. This file controls how Angular CLI builds and serves your application.

3. tsconfig.json

TypeScript compiler configuration:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ES2022",
```

```
"lib": ["ES2022", "dom"],
"strict": true,
"esModuleInterop": true,
"skipLibCheck": true,
"forceConsistentCasingInFileNames": true
}
}
```

Output:

Purpose: Defines how TypeScript code is compiled to JavaScript. Settings like `strict: true` enable type checking to catch errors early.

The src/ Directory Structure

```
src/
├── app/                # Application code
│   ├── app.component.ts # Root component (logic)
│   ├── app.component.html # Root component (template)
│   ├── app.component.css # Root component (styles)
│   ├── app.component.spec.ts # Root component (tests)
│   └── app.module.ts      # Root module
├── assets/             # Static files (images, fonts)
├── environments/       # Environment configurations
├── index.html          # Main HTML file
├── main.ts             # Application entry point
├── polyfills.ts         # Browser polyfills (if needed)
└── styles.css           # Global styles
```

The app/ Folder - Your Application Code

This is where you'll spend most of your development time. Let's examine the root component files:

app.component.ts (Component Logic)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'my-first-app';
}
```

Output:

Explanation:

- `@Component` - Decorator that marks this class as an Angular component
- `selector: 'app-root'` - HTML tag name (`<app-root>`) used in index.html
- `templateUrl` - Path to HTML template
- `styleUrls` - Path to CSS styles
- `title` - Component property available in the template

app.module.ts (Root Module)

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
})
```

```
bootstrap: [AppComponent]
})
export class AppModule { }
```

Output:

Explanation:

- `declarations` - Components, directives, and pipes belonging to this module
- `imports` - Other modules this module depends on
- `providers` - Services available throughout the application
- `bootstrap` - The root component Angular creates and inserts into index.html

Running Your First Application

Let's modify the default application to verify everything is working:

Step 1: Edit app.component.ts

```
export class AppComponent {
  title = 'Welcome to Angular Development!';
  message = 'Your first Angular application is running successfully.';
}
```

Step 2: Edit app.component.html

```
<div style="text-align:center">
  <h1>{{ title }}</h1>
  <p>{{ message }}</p>
</div>
```

Step 3: Run the Application

```
ng serve --open
```

Output:

Browser Display:

Welcome to Angular Development!

Your first Angular application is running successfully.

Result: The application displays your custom title and message. Any changes you make to the files will automatically refresh in the browser (live reload).

Common CLI Commands Summary

Command	Description	Usage
<code>ng new</code>	Create new Angular project	<code>ng new my-app</code>
<code>ng serve</code>	Start development server	<code>ng serve --open</code>
<code>ng build</code>	Build application	<code>ng build --configuration production</code>
<code>ng generate</code>	Generate component/service/etc.	<code>ng generate component header</code>
<code>ng test</code>	Run unit tests	<code>ng test</code>
<code>ng version</code>	Display version information	<code>ng version</code>

Troubleshooting Common Issues

Issue 1: Port Already in Use

Error: Port 4200 is already in use.

Solution: Use a different port:

```
ng serve --port 4300
```

Issue 2: Node Version Compatibility

Error: The engine "node" is incompatible with this module.

Solution: Update Node.js to a compatible version (14.x or higher for most Angular versions).

Issue 3: npm Installation Errors

Error: EACCES: permission denied

Solution: On Linux/Mac, avoid using `sudo`. Instead, configure npm to use a different directory. On Windows, run command prompt as administrator.

Summary

In this section, we covered:

- Installing prerequisites: Node.js and npm
- Installing Angular CLI globally using npm
- Creating Angular applications with `ng new` command and configuration options
- Running applications with `ng serve` and building with `ng build`

- Understanding the Angular bootstrapping process: index.html → main.ts → AppModule → AppComponent
- Exploring the Angular project structure: configuration files and src/ directory
- Key configuration files: package.json, angular.json, and tsconfig.json

With your Angular environment set up, you're now ready to explore development tools and start building Angular applications.

Previous: [1.1 Angular Introduction](#)

Next: [1.3 Development Tools](#)

